

Using Eclipse

(just write code in eclipse all commands to run in terminal)

Eclipse : new -> java project

Make new folder (say src1)

In that folder: new -> file ->(3 files)

In eclipse new java project → keep same java version → disable module (ie create module-info.java file)->finish→create new folder src1

**In same project in src1 folder new→file →paste whole code with .java (for all 3 files)
No need to explicitly mention path same commands!!!**

1. WordCount.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(WordMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path("input.txt"));
        FileOutputFormat.setOutputPath(job, new Path("output"));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

2. WordMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
```

```

    public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
        String[] tokens = value.toString().split("\\s+");
        for (String token : tokens) {
            word.set(token.toLowerCase().replaceAll("[^a-zA-Z]", ""));
            if (!word.toString().isEmpty()) {
                context.write(word, one);
            }
        }
    }
}

```

3. IntSumReducer.java

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
public class IntSumReducer extends Reducer<Text, IntWritable, Text,
IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

```

Save all files

In that project create a untitled file (save as input.txt)

Go to terminal:

hadoop1@aakanksha:~/try\$ start-all.sh

(switch on all nodes)

WARNING: Attempting to start all Apache Hadoop daemons as hadoop1 in 10 seconds.

WARNING: This is not a recommended production deployment configuration.

WARNING: Use CTRL-C to abort.

Starting namenodes on [localhost]

Starting datanodes

Starting secondary namenodes [aakanksha]

Starting resourcemanager

Starting nodemanagers

hadoop1@aakanksha:~\$ cd try

Here try is my folder name where java project is created

hadoop1@aakanksha:~/try\$ ls

bin input.txt src src1

(Just verifying that all things are there

src1 is folder where all 3 java files are saved)

hadoop1@aakanksha:~/try\$ mkdir -p classes

(Creating a folder classes to store compiled .java files if not created then it gets automatically created)

hadoop1@aakanksha:~/try\$ ls

bin classes input.txt src src1

hadoop1@aakanksha:~/try\$ javac -classpath "\$(hadoop classpath)" -d classes
src1/*.java

(to create .class files of all 3 java files

Here I have done error it should be classes not **classses**

)

hadoop1@aakanksha:~/try\$ cd classses

hadoop1@aakanksha:~/try/classses\$ jar -cvf wordcount.jar *.class

added manifest

adding: IntSumReducer.class(in = 1593) (out= 663)(deflated 58%)

adding: WordCount.class(in = 1412) (out= 780)(deflated 44%)

adding: WordMapper.class(in = 1945) (out= 862)(deflated 55%)

(creating jar file)

hadoop1@aakanksha:~/try/classses\$ ls

IntSumReducer.class WordCount.class **wordcount.jar** WordMapper.class

hadoop1@aakanksha:~/try/classses\$ cd ..

hadoop1@aakanksha:~/try\$ hadoop fs -mkdir -p /user/hadoop1

hadoop1@aakanksha:~/try\$ hadoop fs -put input.txt /user/hadoop1

(hadoop1 is user name)

hadoop1@aakanksha:~/try\$ hadoop jar classses/wordcount.jar WordCount input.txt

(output named file gets created)

hadoop jar classes/wordcount.jar WordCount input.txt

output

```
2025-04-25 18:03:34,079 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to
ResourceManager at /127.0.0.1:8032
2025-04-25 18:03:34,310 WARN mapreduce.JobResourceUploader: Hadoop command-line
option parsing not performed. Implement the Tool interface and execute your application with
ToolRunner to remedy this.
2025-04-25 18:03:34,329 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding
for path: /tmp/hadoop-yarn/staging/hadoop1/.staging/job_1745584084518_0001
2025-04-25 18:03:34,494 INFO input.FileInputFormat: Total input files to process : 1
2025-04-25 18:03:34,551 INFO mapreduce.JobSubmitter: number of splits:1
2025-04-25 18:03:34,661 INFO mapreduce.JobSubmitter: Submitting tokens for job:
job_1745584084518_0001
2025-04-25 18:03:34,661 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-04-25 18:03:34,794 INFO conf.Configuration: resource-types.xml not found
2025-04-25 18:03:34,794 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-04-25 18:03:35,178 INFO impl.YarnClientImpl: Submitted application
application_1745584084518_0001
2025-04-25 18:03:35,202 INFO mapreduce.Job: The url to track the job:
http://aakanksha:8088/proxy/application_1745584084518_0001/
2025-04-25 18:03:35,202 INFO mapreduce.Job: Running job: job_1745584084518_0001
2025-04-25 18:03:41,288 INFO mapreduce.Job: Job job_1745584084518_0001 running in uber
mode : false
2025-04-25 18:03:41,290 INFO mapreduce.Job: map 0% reduce 0%
2025-04-25 18:03:44,327 INFO mapreduce.Job: map 100% reduce 0%
2025-04-25 18:03:48,359 INFO mapreduce.Job: map 100% reduce 100%
2025-04-25 18:03:49,384 INFO mapreduce.Job: Job job_1745584084518_0001 completed
successfully
2025-04-25 18:03:49,435 INFO mapreduce.Job: Counters: 54
    File System Counters
        FILE: Number of bytes read=115
        FILE: Number of bytes written=617517
        FILE: Number of read operations=0
        FILE: Number of large read operations=0
        FILE: Number of write operations=0
        HDFS: Number of bytes read=182
        HDFS: Number of bytes written=69
        HDFS: Number of read operations=8
        HDFS: Number of large read operations=0
        HDFS: Number of write operations=2
        HDFS: Number of bytes read erasure-coded=0
    Job Counters
```

Launched map tasks=1
Launched reduce tasks=1
Data-local map tasks=1
Total time spent by all maps in occupied slots (ms)=1491
Total time spent by all reduces in occupied slots (ms)=1698
Total time spent by all map tasks (ms)=1491
Total time spent by all reduce tasks (ms)=1698
Total vcore-milliseconds taken by all map tasks=1491
Total vcore-milliseconds taken by all reduce tasks=1698
Total megabyte-milliseconds taken by all map tasks=1526784
Total megabyte-milliseconds taken by all reduce tasks=1738752

Map-Reduce Framework

Map input records=4
Map output records=14
Map output bytes=130
Map output materialized bytes=115
Input split bytes=109
Combine input records=14
Combine output records=10
Reduce input groups=10
Reduce shuffle bytes=115
Reduce input records=10
Reduce output records=10
Spilled Records=20
Shuffled Maps =1
Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=22
CPU time spent (ms)=1050
Physical memory (bytes) snapshot=599814144
Virtual memory (bytes) snapshot=5491290112
Total committed heap usage (bytes)=520093696
Peak Map Physical memory (bytes)=337338368
Peak Map Virtual memory (bytes)=2740539392
Peak Reduce Physical memory (bytes)=262475776
Peak Reduce Virtual memory (bytes)=2752118784

Shuffle Errors

BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters

Bytes Read=73
File Output Format Counters
Bytes Written=69

hadoop1@aakanksha:~/try\$ hadoop fs -cat /user/hadoop1/output/part-r-00000

(print output from output file)

```
count 1
hadoop      3
hello 2
is 1
of 1
prof 1
run 1
test 1
this 2
word 1
```

When re executing the file input.txt and output files already exists so:

hadoop1@aakanksha:~/try\$ hadoop fs -rm -r /user/hadoop1/input.txt

Deleted /user/hadoop1/input.txt

hadoop1@aakanksha:~/try\$ hadoop fs -rm -r /user/hadoop1/output

Deleted /user/hadoop1/output

hadoop1@aakanksha:~/try\$ hadoop fs -ls /user/hadoop1

(deleted both files and checking it)

hadoop1@aakanksha:~/try\$ stop-all.sh

(switch off all nodes)

WARNING: Stopping all Apache Hadoop daemons as hadoop1 in 10 seconds.

WARNING: Use CTRL-C to abort.

Stopping namenodes on [localhost]

Stopping datanodes

Stopping secondary namenodes [aakanksha]

Stopping nodemanagers

localhost: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill

-9

Stopping resourcemanager

SCALA

Easier one

In text editor:

// Step 1: Read input file from /home/input.txt

val inputFile = "file:///home/aakanksha/SEM_6_Labs/DSBDA/input.txt" **actual path to your input file**

val input = sc.textFile(inputFile)

// Step 2: Perform word count

val words = input.flatMap(line => line.split("\\s+"))

val wordPairs = words.map(word => (word, 1))

val wordCounts = wordPairs.reduceByKey(_ + _)

// Step 3: Save result to /home/output

val outputDir = "file:///home/aakanksha/SEM_6_Labs/DSBDA/output" **actual path where u want output**

wordCounts.coalesce(1).saveAsTextFile(outputDir)

(Save as any_name.scala)

Terminal:

aakanksha@aakanksha:~\$ spark-shell

(start spark shell)

25/04/25 18:35:23 WARN Utils: Your hostname, aakanksha resolves to a loopback address: 127.0.1.1; using 192.168.1.4 instead (on interface wlp0s20f3)

25/04/25 18:35:23 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".

To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to

```

      _____
     /  _/  _/  _/  _/  _/  _/
    _\V _V _\V _\V _\V _\V _\V
   /  _/  _/  _/  _/  _/  _/  _/
  /  _/  _/  _/  _/  _/  _/  _/
 /  _/  _/  _/  _/  _/  _/  _/
/  _/  _/  _/  _/  _/  _/  _/

```

version 3.4.1

Using Scala version 2.13.8 (OpenJDK 64-Bit Server VM, Java 11.0.27)

Type in expressions to have them evaluated.

Type :help for more information.

25/04/25 18:35:26 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

Spark context Web UI available at http://192.168.1.4:4040

Spark context available as 'sc' (master = local[*], app id = local-1745586327837).

Spark session available as 'spark'.

scala> :load try.scala (any_name.scala)

val args: Array[String] = Array()

Loading try.scala...

val inputFile: String = file:///home/aakanksha/SEM_6_Labs/DSBDA/input.txt

val input: org.apache.spark.rdd.RDD[String] =

file:///home/aakanksha/SEM_6_Labs/DSBDA/input.txt MapPartitionsRDD[1] at textFile at try.scala:1

val words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at flatMap at try.scala:1

val wordPairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at map at try.scala:1

val wordCounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduceByKey at try.scala:1

val outputDir: String = file:///home/aakanksha/SEM_6_Labs/DSBDA/output

scala> :quit